PYTHON IN PHYSICS

1.What is Python Implementation in Physics?

Python implementation in physics means using the Python programming language to solve physics problems, perform calculations, simulate physical systems, analyze experimental data, and visualize scientific results.

Instead of doing long manual calculations, physicists use Python to model motion, waves, electricity, quantum systems, and real-world physical phenomena efficiently.

It is widely used in:

- Research laboratories
- Engineering & space science
- Astronomy & astrophysics
- Computational physics
- STEM education

2. Why Python is Used in Physics?

Numerical Calculations

- Velocity
- Force
- Energy
- Motion equations

Simulation

- Projectile motion
- Wave motion
- Planetary motion

Data Analysis

- Experimental measurements
- Lab results

Graph Visualization

- Motion graphs
- Oscillation graphs

AI in Physics Research

- Pattern detection
- Predictive modeling

3. Python Libraries Used in Physics

Library	Use
NumPy	Mathematical calculations
Matplotlib	Graph plotting
SciPy	Scientific computing
SymPy	Symbolic mathematics
Pandas	Data analysis

Program 1 — Velocity Calculation

```python
distance = float(input("Enter Distance: "))
time = float(input("Enter Time: "))

velocity = distance / time
print("Velocity =", velocity)
```

Program 2 — Newton's Second Law (Force)

```python
mass = float(input("Enter Mass: "))
acceleration = float(input("Enter Acceleration: "))

force = mass * acceleration
print("Force =", force)
```

Program 3 — Ohm's Law

```python
voltage = float(input("Enter Voltage: "))
resistance = float(input("Enter Resistance: "))

current = voltage / resistance
print("Current =", current)
```

Program 4 — Free Fall Motion

```python
g = 9.8
time = float(input("Enter Time: "))

distance = 0.5 * g * time**2
print("Distance Fallen =", distance)
```

Program 5 — Motion Graph

```python
import matplotlib.pyplot as plt

time = [0,1,2,3,4]
distance = [0,5,20,45,80]

plt.plot(time, distance)
plt.xlabel("Time")
plt.ylabel("Distance")
plt.title("Motion Graph")
plt.show()
```

Program 6-Acceleration Calculation

```
u = float(input("Enter Initial Velocity: "))
v = float(input("Enter Final Velocity: "))
t = float(input("Enter Time: "))

a = (v - u) / t
print("Acceleration =", a)
```

Program 7 - Density Calculation

```python
mass = float(input("Enter Mass: "))
volume = float(input("Enter Volume: "))

density = mass / volume
print("Density =", density)
```

Program 8- Pressure Calculation

```python
force = float(input("Enter Force: "))
area = float(input("Enter Area: "))

pressure = force / area
print("Pressure =", pressure)
```

Program 9- Simple Pendulum Time Period

```python
import math

L = float(input("Enter Length: "))
g = 9.8

T = 2 * math.pi * math.sqrt(L/g)
print("Time Period =", T)
```

Program 10 – Numerical Integration

```python
import numpy as np

x = np.linspace(0,10,100)
y = x**2

area = np.trapz(y,x)
print("Area Under Curve =", area)
```

Program 11 - Damped Oscillation Simulation

```python
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,10,100)
x = np.exp(-0.2*t) * np.cos(2*t)

plt.plot(t,x)
plt.title("Damped Oscillation")
plt.show()
```

Program 12 - Projectile Motion Simulation

```python
import numpy as np

g = 9.8
u = float(input("Enter Initial Velocity: "))
angle = 45

t = np.linspace(0,5,50)
x = u*np.cos(np.radians(angle))*t
y = u*np.sin(np.radians(angle))*t - 0.5*g*t**2

print("X Positions:", x)
print("Y Positions:", y)
```

Program 13 - Fourier Wave Analysis

```python
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,1,500)
wave = np.sin(2*np.pi*5*t)

plt.plot(t,wave)
plt.title("Wave Signal")
plt.show()
```

Program 14 - Electric Field Calculation

```python
k = 9*10**9

q = float(input("Enter Charge: "))
r = float(input("Enter Distance: "))

E = k*q/(r**2)
print("Electric Field =", E)
```

Program 15 - Gravitational Force

```python
# Newton's Law of Gravitation
G = 6.67e-11
m1 = 5.97e24   # Earth mass
m2 = 7.35e22   # Moon mass
r = 3.84e8     # distance

F = G*m1*m2/(r*r)
print("Gravitational Force:", F)
```

MINI PROJECT

Project Title

Quantum Particle in a Box Simulation using Python

Aim

To simulate the wavefunction and probability density of a quantum particle confined in a one-dimensional box using Python.

Theory

In quantum mechanics, a particle confined in a box has discrete energy levels.
The wavefunction is:

$\psi(x) = \sqrt{(2/L)} \sin(n\pi x/L)$

Where:

- n = quantum number
- L = length of box
- x = position

Energy levels:

□ $E_n \propto n^2$

This project visualizes:

- Wavefunction
- Probability density

Algorithm

1. Import required libraries
2. Define box length
3. Get quantum number input
4. Create position array
5. Calculate wavefunction
6. Compute probability density
7. Plot graphs

python

```python
import numpy as np
import matplotlib.pyplot as plt

print("Quantum Particle in a Box Simulation")

# Box Length
L = float(input("Enter Length of Box: "))

# Quantum Number
n = int(input("Enter Quantum Number: "))

# Position values
x = np.linspace(0, L, 500)
```

```python
# Wavefunction
psi = np.sqrt(2/L) * np.sin(n * np.pi * x / L)

# Probability Density
probability = psi**2

# Plot Wavefunction
plt.plot(x, psi)
plt.title("Wavefunction")
plt.xlabel("Position")
plt.ylabel("Psi(x)")
plt.show()

# Plot Probability Density
plt.plot(x, probability)
plt.title("Probability Density")
plt.xlabel("Position")
plt.ylabel("Psi^2")
plt.show()
```